

GraphOne Intelligence Graph

Architecture & Production Design

1. Scale Strategy — Collecting 500,000+ Records Without Manual Intervention

The pipeline is built around three principles that let it scale via configuration and worker count, not code changes:

- Config-driven concurrency: every scraper draws from a single `AsyncHttpClient` wrapping an `asyncio.Semaphore` whose limit is a config value (`SCRAPER_MAX_CONCURRENCY`). Raising it from 20 to 200 across a fleet of workers is an env var change.
- Horizontal partitioning: each source is naturally partitionable — arXiv by category (`cs.AI`, `cs.LG`, `cs.CL`, ...) and date range; job boards by category/page offset; news by source domain. A driver process assigns partitions to worker processes/containers, each running the identical scraper code against a distinct slice.
- Stateless workers, shared coordination: workers hold no local state beyond an in-flight batch. Coordination (what's been claimed, what's been seen) lives in shared infrastructure — a work queue (SQS/Redis Streams) for partition assignment and a shared dedupe store (see §3) for freshness. This means workers can be killed and restarted, or added mid-run, without corrupting progress.
- Orchestration: in production this runs as a scheduled fleet (e.g. Kubernetes CronJob spinning up N worker Pods, or Airflow/Prefect DAG with dynamic task mapping over partitions) rather than a single long-lived process — a single node has no chance of reaching 500k records in reasonable time regardless of code quality.

Concretely, going from the trial's ~1-2k records to 500k means: (a) raise concurrency, (b) partition each vertical into more slices, (c) run more worker replicas, (d) point dedupe/state at Redis instead of local SQLite. No scraper, parser, or schema code changes.

2. Handling 413s and 429s Across Thousands of Concurrent Extractions

429 (Rate Limited)

- Every HTTP call (scrapping and LLM) goes through a shared retry wrapper that reads `Retry-After` when present, else computes exponential backoff with jitter: $\min(\text{base} * 2^{\text{attempt}}, \text{cap}) + \text{random}(0, 30\% \text{ of that value})$. Jitter is essential at scale — without it, thousands of workers backing off on the same fixed schedule re-hit the target in synchronized waves (thundering herd), which is worse than no backoff at all.
- For the LLM chain specifically, a 429 on the current tier triggers a bounded number of same-tier retries (worth it — that tier is usually cheapest), then falls through to the next tier in the chain (Gemini → Groq → DeepSeek) rather than blocking the whole pipeline on one provider's rate limit.

413 (Payload Too Large)

- Handled proactively, not reactively: before any LLM call, input text is measured against a token budget (chars/4 heuristic) and pre-chunked on paragraph boundaries (falling back to sentence boundaries for oversized single paragraphs) if it exceeds the budget. This means a 413 should rarely occur in the first place.
- Chunks carry a configurable token overlap so an entity or fact split across a chunk boundary still appears whole in at least one chunk — avoiding silent data loss at chunk edges.
- If a 413 occurs anyway (e.g. an unusually verbose provider-side system prompt pushes over budget), that's treated as a tier failure and the extraction falls through to the next tier in the chain, which may have a larger context window.

3. Freshness Tracking — Never Processing the Same Item Twice Across Distributed Nodes

Trial implementation: a SQLite table keyed on `SHA-256(normalized URL)`, checked before enqueueing and written immediately after a successful fetch. Fine for a single-process run.

Production (distributed nodes): the identical interface (`seen(url)`, `mark_seen(url)`) is backed by Redis instead of SQLite:

- `mark_seen` uses SET key NX EX <ttl> — atomic check-and-set, so two workers racing on the same URL can't both 'win'; only one proceeds.
- A TTL (e.g. 30 days) bounds memory growth and models the fact that 'freshness' has a natural expiry — we don't need to remember a URL forever, only long enough to guarantee no duplicate processing within the freshness window the business cares about.
- For the strict 24-hour freshness requirement specifically: publish timestamps are normalized (`src/utls/date_parser.py`) from either a strict source timestamp (RSS/API) or parsed relative text ('2 hours ago'). When a source provides neither, we fall back to the dedupe store itself as a heuristic — if a URL's hash was not seen on the previous crawl run (whose timestamp we know), and it now appears, we treat it as new since last run, which is the 'intelligent heuristic' the brief asks for when a source lacks a strict date.
- Because the dedupe backend is swapped behind one unchanged interface, moving from single-node SQLite to distributed Redis touches exactly one file.

4. Storage Strategy

Primary datastore: PostgreSQL

Reasoning: the canonical schemas (Startup, Product, Paper, Job, News) are semi-structured but have a stable core shape known up front — a relational store with JSONB columns for the flexible `content` payload gives us the best of both worlds: strict typing and indexing on fields we always query (recordType, source.url, collectedAt, canonical entity id) and schema flexibility inside `content` for per-type variation, without needing a schema migration every time a new optional field appears. Postgres's JSONB indexing (GIN indexes) makes this performant, not just convenient.

Relationship / graph layer: Neo4j (or Amazon Neptune)

The product is literally called an 'Intelligence Graph' — the value is in the relationships (startup → product → paper → repo → founder → job), not just the entities in isolation. A property graph database is the natural fit for queries like 'show me all papers connected to startups that posted an ML job in the last 24h,' which are multi-hop traversals that are painful in pure relational joins at scale. Postgres remains the system of record for raw ingested data and audit trail; the graph layer is built from it as a derived, queryable view optimized for traversal — not the primary write target, so a graph-rebuild is always possible from Postgres if the graph store needs to be reconstructed.

Vector layer: pgvector (Postgres extension) initially, dedicated vector DB (Pinecone/Weaviate) at scale

For semantic entity resolution and 'similar paper / similar startup' search beyond what fuzzy string matching catches. Starting inside Postgres (pgvector) avoids a fourth system to operate for the trial-to-early-production stage; migrating to a dedicated vector store becomes worth the operational overhead once embedding volume and query latency requirements outgrow what pgvector comfortably serves.

Object storage: S3-compatible bucket

Raw HTML/PDF snapshots at time of scrape are stored here, referenced by URL from the Postgres row. This is what makes 'every record must trace back to a legitimate, valid source URL' auditable after the fact even if the live page later changes or disappears — we keep evidence of what we actually saw.